Análise Comparativa de Estratégias de Controle Populacional em Algoritmos Genéticos aplicados ao Problema das N-Rainhas

Vinícius Rubin[1], **Igor Yepes**[1]

[1]Instituto de Educação, Ciência e Tecnologia Farroupilha - Campus Frederico Westphalen (IFFAR-FW)

Abstract. *This article investigates how different population control strategies affect the performance of genetic algorithms when solving the n-queens problem. We describe the design of a baseline implementation, outline the experimental protocol, and discuss how metrics such as execution time, number of generations, and success rate will be interpreted. The study aims to identify configurations that preserve diversity while accelerating convergence, contributing guidelines for practitioners who deploy evolutionary heuristics in combinatorial search tasks.*

Resumo. *Este artigo investiga como diferentes estratégias de controle populacional influenciam o desempenho de algoritmos genéticos na resolução do problema das n-rainhas. São descritos o planejamento da implementação base, o protocolo experimental e a forma de análise de métricas como tempo de execução, número de gerações e taxa de sucesso. O estudo busca apontar configurações que preservem a diversidade e acelerem a convergência, oferecendo subsídios para o emprego de heurísticas evolutivas em tarefas de busca combinatória.*

1. Introdução

Algoritmos Genéticos (AGs) são heurísticas evolutivas baseadas em seleção natural, amplamente utilizadas para otimização em espaços de busca complexos [Eiben and Smith 2015]. A capacidade de combinar exploração estocástica com operadores simples de reprodução torna esses algoritmos versáteis em aplicações que vão de problemas combinatórios a tarefas de engenharia e inteligência artificial.

Entre os elementos que influenciam o desempenho dos AGs, o *controle populacional* — isto é, o modo como indivíduos são mantidos, removidos ou substituídos ao longo das gerações — desempenha papel central. Estratégias como substituição total, substituição parcial, elitismo e abordagens baseadas em ciclos de vida têm sido utilizadas na prática, mas a literatura apresenta resultados heterogêneos, conduzidos em contextos experimentais distintos e com ausência de padronização. Como consequência, ainda é difícil comparar, de forma direta e controlada, o impacto real de cada mecanismo no comportamento do algoritmo.

Essa lacuna motiva o presente estudo, cujo objetivo é **comparar sistematicamente diferentes estratégias de controle populacional em um mesmo cenário experimental**, mantendo constantes todos os demais componentes do AG (representação, seleção, cruzamento e mutação). Para isso, utiliza-se o problema das n-rainhas como

ambiente de teste, um benchmark clássico pela sua estrutura simples, escalabilidade controlada e ampla adoção em estudos de otimização.

A contribuição deste trabalho é dupla: (i) oferecer uma avaliação empírica rigorosa de quatro mecanismos de controle populacional — substituição total, substituição parcial, elitismo e uma abordagem baseada em idade — sob condições experimentais uniformes; e (ii) analisar como cada estratégia afeta taxa de sucesso, esforço evolutivo e custo computacional à medida que a dificuldade do problema aumenta.

Os experimentos são conduzidos com múltiplas execuções independentes para cada configuração, permitindo estimar médias, variâncias e intervalos de confiança, além de aplicar testes estatísticos para comparação entre estratégias. Essa estrutura experimental fornece evidências quantitativas que auxiliam na escolha de mecanismos mais adequados para AGs aplicados a problemas combinatórios semelhantes.

2. Fundamentação Teórica

2.1. Algoritmos Genéticos

Algoritmos Genéticos (AGs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores de seleção, cruzamento e mutação [Eiben and Smith 2015, Holland 1975]. Cada indivíduo é avaliado por uma função de aptidão (*fitness*), e o processo evolutivo tende a favorecer configurações de melhor desempenho.

O comportamento de um AG é fortemente influenciado por três grupos de decisões: (i) representação dos indivíduos, (ii) função de avaliação e (iii) desenho dos operadores e da política de substituição [de Lacerda and de Leon Ferreira de Carvalho 1999]. Enquanto representação e operadores de cruzamento/mutação são amplamente discutidos na literatura, o *controle populacional* — isto é, a forma como a população é renovada e mantida ao longo das gerações — recebe tratamento menos sistemático, embora esteja diretamente ligado à diversidade, à convergência e ao risco de aprisionamento em ótimos locais.

Neste trabalho, representação, função de aptidão, seleção, cruzamento e mutação são mantidos fixos, e o foco recai justamente sobre o componente de controle populacional, comparando diferentes mecanismos de substituição em um cenário experimental comum.

2.2. O Problema das *n*-Rainhas

O problema das *n*-rainhas consiste em posicionar *n* rainhas em um tabuleiro de xadrez de dimensões *n*×*n* de forma que nenhuma delas ataque outra, o que implica ausência de conflitos em linhas, colunas e diagonais [Bell and Stevens 2009]. Apesar da descrição simples, o número de configurações possíveis cresce rapidamente com *n*, tornando abordagens exaustivas impraticáveis para valores mais elevados.

Estudos como o de Ali et al. [Ali et al. 2023] mostram que técnicas clássicas de busca (BFS, DFS) são eficazes apenas para instâncias pequenas, perdendo viabilidade à medida que o tabuleiro cresce. Em contrapartida, o problema possui estrutura bem definida, múltiplas soluções conhecidas e um parâmetro de dificuldade (n) facilmente ajustável. Por isso, é amplamente adotado como *benchmark* em experimentos com

heurísticas e meta-heurísticas, incluindo AGs [Sarkar and Nag 2017, Farhan et al. 2015, Ali et al. 2023].

Neste estudo, o problema das *n*-rainhas é utilizado como ambiente de teste justamente por combinar simplicidade conceitual, escalabilidade controlada e ampla presença na literatura, o que facilita a comparação dos resultados obtidos com trabalhos anteriores.

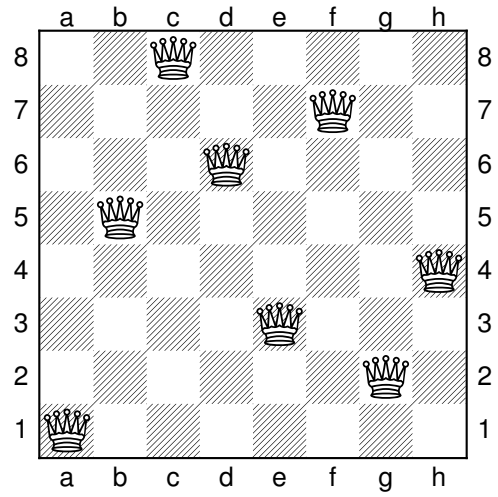


Figura 1. Exemplo de solução válida para o problema das 8-rainhas.

2.3. Uso da Linguagem Python e Bibliotecas Relacionadas

A implementação dos experimentos é realizada em `Python`, linguagem amplamente utilizada em computação científica devido à clareza sintática e à disponibilidade de bibliotecas numéricas e de visualização [Python Software Foundation 2025a]. A biblioteca `NumPy` é empregada para representar populações e operar sobre vetores e matrizes de forma vetorizada [Harris et al. 2020], enquanto o gerador `numpy.random` é utilizado nas operações estocásticas de inicialização, seleção e aplicação de operadores [Python Software Foundation 2025b, Harris et al. 2020]. A medição de tempo é feita com o módulo `time` [Python Software Foundation 2025c], e a síntese gráfica dos resultados é conduzida com `matplotlib` [Hunter 2007].

Esse conjunto de ferramentas oferece suporte suficiente para implementar o AG base, alternar entre diferentes estratégias de controle populacional e registrar métricas de desempenho de forma reprodutível, evitando que questões de infraestrutura ofusquem o foco principal do estudo.

2.4. Trabalhos Relacionados

Diversos trabalhos utilizam o problema das *n*-rainhas como *benchmark* para avaliar variações em AGs, explorando principalmente representação e controle de diversidade. Nesta subseção, destacam-se estudos que motivam o desenho experimental adotado aqui e ajudam a situar a contribuição deste trabalho, que se volta especificamente ao impacto de diferentes estratégias de controle populacional.

2.4.1. Representações Binária e Inteira para o Problema das *n*-Rainhas [Richter and Berlezi 2012]

Richter et al. [Richter and Berlezi 2012] comparam duas representações genéticas para o problema das *n*-rainhas: uma codificação binária e uma codificação inteira do tipo *InOrder*. A primeira exige conversão de bits para posições de rainhas e é suscetível a colisões em linhas; a segunda mapeia diretamente cada coluna a uma linha distinta, evitando conflitos de linha e coluna por construção.

Os resultados mostram que a representação *InOrder* reduz o número de soluções inviáveis e acelera a convergência, sobretudo para tabuleiros maiores. Embora os autores mantenham o controle populacional em um esquema tradicional, seu trabalho reforça a importância de fixar uma representação adequada antes de comparar outras variações de um AG. Neste estudo, adotamos uma representação por permutação semelhante à *InOrder* justamente para minimizar o impacto de soluções inviáveis e isolar o efeito das estratégias de substituição populacional.

2.4.2. Comparação entre Algoritmo Genético e Buscas Clássicas (BFS/DFS) [Ali et al. 2023]

Ali et al. [Ali et al. 2023] comparam um AG com buscas clássicas (BFS/DFS) no problema das *n*-rainhas, analisando a escalabilidade de cada método. Utilizando uma representação vetorial em que cada posição do cromossomo indica a linha ocupada por uma rainha, os autores observam que, para instâncias pequenas, BFS e DFS são mais eficientes em tempo de execução, mas o AG passa a ser mais competitivo à medida que o tamanho do tabuleiro aumenta.

Embora o foco do estudo seja a comparação entre paradigmas (busca exaustiva vs. AG), os resultados reforçam o papel dos AGs como alternativa viável para instâncias maiores e justificam seu uso como meta-heurística de referência neste trabalho. Diferentemente de Ali et al., aqui não comparamos AGs com métodos clássicos de busca, mas exploramos, dentro do próprio paradigma evolutivo, como diferentes políticas de controle populacional afetam desempenho em um problema combinatório já consolidado na literatura.

2.4.3. Algoritmo Genético Adaptativo com Controle de Diversidade [Sarkar and Nag 2017]

Sarkar e Nag [Sarkar and Nag 2017] propõem um AG adaptativo para o problema das *n*-rainhas, enfatizando mecanismos explícitos de controle da diversidade populacional. Os autores utilizam uma representação por permutação, um cruzamento do tipo *Order 1 Crossover* e operadores de mutação com probabilidades relativamente altas, além de estruturas auxiliares (como um repositório de indivíduos menos aptos) para evitar estagnação em ótimos locais.

Os resultados indicam que o esquema adaptativo encontra soluções ótimas para valores elevados de N (até 100), sugerindo que a forma como a população é renovada

tem influência direta na qualidade e na velocidade da convergência. O trabalho, porém, combina várias alterações simultâneas (função de aptidão, mutação, repositório, etc.), o que dificulta isolar o efeito de cada componente. Em contraste, o presente estudo mantém constantes representação e operadores genéticos e varia apenas a política de controle populacional, buscando medir de forma mais direta o impacto da substituição e do envelhecimento sobre o desempenho do AG.

2.4.4. Busca de Todas as Soluções Válidas para as 8-Rainhas [Farhan et al. 2015]

Farhan et al. [Farhan et al. 2015] utilizam um AG para encontrar as 92 soluções válidas do problema das 8-rainhas, empregando uma representação vetorial que elimina conflitos em linhas e colunas e concentra a avaliação em ataques diagonais. A função de aptidão é definida em termos do número de conflitos diagonais, e o AG é executado até que todas as soluções distintas sejam encontradas.

Embora o objetivo seja diferente do deste trabalho (cobertura do espaço de soluções em vez de análise comparativa de políticas de controle), o estudo ilustra a capacidade dos AGs de explorar um espaço combinatório rico e de encontrar múltiplas soluções válidas. Aqui, aproveitamos essa mesma característica do problema, mas direcionamos a atenção para como diferentes estratégias de substituição e um esquema de controle por idade alteram a taxa de sucesso e o esforço necessário para encontrar ao menos uma solução válida em instâncias progressivamente mais difíceis.

2.5. Controle Populacional Baseado em Idade

Entre as estratégias possíveis de controle populacional, abordagens que incorporam algum tipo de envelhecimento ou tempo de vida vêm sendo exploradas como forma de equilibrar exploração e intensificação, evitando que indivíduos muito antigos dominem a população indefinidamente. O controle por idade adotado neste trabalho segue essa perspectiva, mas em uma forma simples e acoplada diretamente à aptidão atual dos indivíduos.

Cada indivíduo possui uma idade e um *lifetime* (tempo máximo de vida). A cada geração, a idade é incrementada em uma unidade; quando a idade ultrapassa o lifetime, o indivíduo é removido. O tamanho da população é mantido fixo: após a remoção dos indivíduos cujo tempo de vida se esgotou, novos descendentes são gerados para preencher as vagas.

2.5.1. Cálculo do Lifetime

O tempo de vida máximo é recalculado a cada geração com base na distribuição de aptidões daquela população. Seja $f_{\min}$ a melhor aptidão da geração, $f_{\max}$ a pior e $f(i)$ a aptidão do indivíduo i. O lifetime é dado por:

$$\text{lifetime}(i) = L_{\min} + \left\lfloor (L_{\max} - L_{\min}) \cdot \frac{f_{\max} - f(i)}{f_{\max} - f_{\min}} \right\rfloor,$$

de modo que indivíduos mais aptos tendem a viver por mais gerações que in-

divíduos menos aptos. Os valores utilizados neste trabalho são

$$L_{\min} = 1, \qquad L_{\max} = 5,$$

escolhidos empiricamente para induzir renovação contínua sem tornar a população excessivamente instável.

2.5.2. Reprodução e Manutenção do Tamanho Populacional

Após a remoção dos indivíduos cujo tempo de vida se esgotou, seja S o número de sobreviventes. O número de descendentes a serem criados é definido por

$$N_{\text{novos}} = \lceil \text{taxa}_{\text{reprodução}} \cdot S \rceil ,$$

com

$$\text{taxa}_{\text{reprodução}} = 0{,}5.$$

Os novos indivíduos são gerados a partir dos operadores padrão do AG (seleção em torneio binário, cruzamento OX e mutação por troca), iniciam com idade zero e têm seus lifetimes calculados após a avaliação. Se a combinação de sobreviventes e descendentes ainda não atingir o tamanho populacional nominal, indivíduos adicionais são reinicializados aleatoriamente.

2.5.3. Resumo e Relação com Este Estudo

O mecanismo de idade adotado pode ser resumido em três passos: (i) incrementar a idade de todos os indivíduos; (ii) remover aqueles cuja idade exceda o lifetime; (iii) gerar novos descendentes até restaurar o tamanho populacional. Ao vincular o lifetime à aptidão relativa de cada geração, o algoritmo promove uma renovação constante, preservando bons indivíduos por mais tempo, mas evitando que qualquer cromossomo permaneça indefinidamente na população.

Essa estratégia fornece um contraste direto com os esquemas de substituição total, substituição parcial e elitismo investigados neste trabalho, permitindo comparar, sob as mesmas condições experimentais, um controle populacional baseado em idade com políticas clássicas que não incorporam envelhecimento explícito. A metodologia detalhada na Seção 3 parte dessa fundamentação para definir os cenários de comparação empírica entre as quatro estratégias.

3. Metodologia

Este estudo compara o impacto de quatro estratégias de controle populacional em algoritmos genéticos (AGs) aplicados ao problema das *n*-rainhas, seguindo diretrizes de experimentação reprodutível em computação evolutiva [Eiben and Smith 2015]. Em todas as configurações, representação, função de aptidão e operadores genéticos permanecem fixos, permitindo isolar o efeito da política de substituição.

3.1. Algoritmo Genético Base

Cada indivíduo é representado por uma permutação de tamanho n, eliminando conflitos em linhas e colunas. A aptidão corresponde ao número de pares de rainhas em conflito diagonal, método de avaliação comum na literatura e com complexidade $O(n^2)$.

A população inicial é gerada embaralhando sequências de 0 a $n-1$. A evolução utiliza seleção por torneio binário, cruzamento *Order Crossover* (OX) e mutação por troca de duas posições. A taxa de mutação é mantida fixa em **10%** em todas as estratégias, reduzindo a dimensionalidade do espaço experimental e possibilitando concentrar a análise exclusivamente nos mecanismos de controle populacional.

3.2. Estratégias de Controle Populacional

Quatro mecanismos são comparados:

- **Substituição total**: toda a população é renovada a cada geração.
- **Substituição parcial**: uma fração dos indivíduos é preservada e o restante substituído.
- **Elitismo**: uma porcentagem dos melhores indivíduos é garantida na nova geração.
- **Controle por idade**: indivíduos possuem tempo de vida limitado, calculado a partir da aptidão, sendo removidos quando ultrapassam seu *lifetime*, e a população pode crescer ou encolher dinamicamente ao longo das gerações.

Os tamanhos populacionais avaliados (100, 200, 300) representam três escalas típicas na prática de AGs — pequena, média e maior — permitindo observar como a quantidade de indivíduos interage com cada política de substituição nas estratégias de população fixa. No caso do controle por idade, esses valores são utilizados como tamanho inicial da população, que passa a variar dinamicamente durante a execução.

A Tabela 1 resume os parâmetros específicos de cada mecanismo.

Tabela 1. Configurações avaliadas por estratégia de controle populacional

Estratégia	Parâmetros	Valores
Substituição total	Tamanho da população	100, 200, 300
	Taxa de mutação	10%
Substituição parcial	Fração substituída	25%, 50%, 75%
	Tamanho da população	100, 200, 300
Elitismo	Fração de elite	10%, 20%, 30%
	Tamanho da população	100, 200, 300
Controle por idade	Lifetime	$L_{\min} = 1$, $L_{\max} = 5$
	Taxa de reprodução	0,5
	Tamanho inicial da população	100, 200, 300

3.3. Controle por Idade no Contexto Experimental

O mecanismo utilizado neste trabalho segue o modelo descrito na Seção 2. Após a avaliação da população, o *lifetime* de cada indivíduo é recalculado por uma função bilinear em função do intervalo de aptidões; a idade é incrementada em uma unidade; e indivíduos cuja idade ultrapassa o seu *lifetime* são removidos.

Seja S o número de sobreviventes após essa etapa. A partir deles, são gerados novos indivíduos por cruzamento, de acordo com uma taxa de reprodução fixa $\tau = 0{,}5$:

$$N_{\text{novos}} = \lfloor \tau \cdot S \rceil \,,$$

onde $\lfloor \cdot \rceil$ denota arredondamento para o inteiro mais próximo. A nova população passa a ter tamanho

$$\text{Pop}(t+1) = S + N_{\text{novos}},$$

permitindo que a população aumente ou diminua ao longo do tempo, de acordo com a dinâmica de sobrevivência e reprodução. Representação, seleção, cruzamento e mutação permanecem idênticos aos das demais estratégias.

Caso todos os indivíduos sejam eliminados em uma geração ($S = 0$), é aplicada uma reinicialização completa da população, gerando um novo conjunto de indivíduos aleatórios com o mesmo tamanho inicial, de modo a evitar estagnação irrecuperável.

3.4. Experimentos

Para cada combinação de:

- estratégia de controle populacional,
- tamanho da população inicial (100, 200, 300),

são realizadas **100 execuções independentes** para cada valor de n. A taxa de mutação é mantida fixa em **10%** em todas as condições experimentais, seguindo recomendações da literatura para representações permutacionais e reduzindo o volume total de cenários, o que permite concentrar a análise exclusivamente no impacto das estratégias de controle populacional.

Critério de parada

Cada execução é encerrada quando:

- uma solução válida (aptidão zero) é encontrada; ou
- o algoritmo atinge o limite de **1.000 gerações**; ou
- o tempo de execução atinge um limite máximo pré-definido $T_{\max}$ = 10 s (limite de tempo por execução).

O valor de $T_{\max}$ é mantido constante para todas as estratégias e configurações de tamanho de população, funcionando como uma barreira computacional uniforme. Esse controle adicional é particularmente relevante para o mecanismo de controle por idade, cujo custo computacional por geração é maior devido à gestão dinâmica de idade, *lifetime* e tamanho populacional. Assim, evita-se que execuções muito lentas distorçam as métricas de tempo e esforço evolutivo.

Métricas coletadas

Para cada execução, registram-se:

- **Tempo de execução total**;
- **Número de gerações até a solução ou até o limite**;
- **Indicador de sucesso** (solução encontrada antes de atingir qualquer limite);
- **Critério de parada acionado** (solução, limite de gerações ou limite de tempo).

3.5. Ambiente de Execução e Análise Estatística

Os experimentos são implementados em `Python` utilizando `numpy`, `numba`, `matplotlib`, `argparse` e `psutil`. A geração de aleatoriedade utiliza `numpy.random.default_rng`, com registro explícito da semente para garantir reprodutibilidade. As análises estatísticas consideram tanto medidas de desempenho (tempo, gerações, taxa de sucesso) quanto o impacto do critério de parada por tempo sobre cada estratégia de controle populacional.

4. Desenvolvimento

Esta seção descreve como o estudo foi concretamente implementado, desde a organização da ferramenta experimental até a execução dos experimentos e a preparação dos dados que embasam a análise apresentada na seção de Resultados e Discussão.

4.1. Arquitetura da Ferramenta Experimental

A implementação foi estruturada como uma pequena *ferramenta de experimentação* em `Python`, organizada em módulos com responsabilidades bem definidas. Essa separação favorece a reprodutibilidade do estudo e permite que novos cenários de teste sejam adicionados com o mínimo de alterações no código existente.

De forma geral, a arquitetura é composta pelos seguintes blocos lógicos:

- **Núcleo do algoritmo genético**: responsável pelo laço evolutivo principal, inicialização da população, aplicação dos operadores genéticos e atualização da geração corrente.
- **Módulo de representação e aptidão**: encapsula a codificação em forma de permutação para o problema das n-rainhas e a função de avaliação baseada na contagem de pares de rainhas em conflito diagonal, conforme definido na Seção 2.
- **Módulo de operadores genéticos**: implementa seleção, cruzamento e mutação de forma independente do problema, recebendo apenas indivíduos e retornando descendentes.
- **Módulo de controle populacional**: concentra a lógica das quatro estratégias de substituição comparadas neste trabalho (população fixa, substituição geracional, elitismo e controle por idade/variável).
- **Módulo de experimentos**: responsável por ler as configurações de cada cenário (tamanho do tabuleiro, limites de gerações e tempo, número de execuções independentes, entre outros), orquestrar a execução repetida do algoritmo genético e registrar as métricas de interesse em arquivos de saída.
- **Módulo de registro e persistência**: implementa a escrita de logs e a serialização dos resultados em formato tabular (`.csv`), de modo a facilitar o pós-processamento estatístico.

Essa organização modular permite que o mesmo núcleo de algoritmo genético seja utilizado com diferentes políticas de controle populacional, modificando-se apenas a configuração carregada pelo módulo de experimentos. Além disso, a presença de um módulo dedicado a registro de resultados simplifica a automatização de baterias extensas de testes, uma vez que a ferramenta pode ser executada em modo não interativo a partir de linha de comando.

4.2. Implementação dos Operadores Genéticos

A implementação segue as decisões de projeto descritas na Seção 3. Cada indivíduo é representado por um vetor de inteiros de tamanho n, onde a posição do vetor corresponde à coluna do tabuleiro e o valor armazenado indica a linha ocupada pela rainha naquela coluna. Como essa representação garante que não haja conflitos em linhas e colunas, a função de aptidão concentra-se em contabilizar apenas conflitos diagonais.

A função de aptidão percorre pares de colunas e verifica se a diferença de índices é igual à diferença absoluta entre as linhas correspondentes. Quando essa condição se verifica, o par de rainhas está em conflito diagonal e uma penalização é aplicada. Para fins de comparação entre estratégias de controle populacional, o valor de aptidão é calculado de forma consistente em todos os cenários, e uma solução ótima é caracterizada pela ausência de conflitos diagonais.

A seleção é realizada por torneio binário: dois indivíduos são escolhidos aleatoriamente da população e o de melhor aptidão é selecionado como pai. Esse procedimento é repetido até que seja formada a quantidade necessária de pais para gerar a prole da próxima geração. Esse método é simples, eficiente e não requer normalização explícita dos valores de aptidão.

O cruzamento adotado é o *Order Crossover* (OX), adequado a representações permutacionais. Na implementação, dois pontos de corte são sorteados aleatoriamente ao longo do vetor. O segmento interno entre os cortes é copiado diretamente de um dos pais para o descendente, preservando a ordem relativa dos elementos. Em seguida, as posições restantes são preenchidas com os genes do outro pai, respeitando a ordem em que aparecem e evitando duplicações. O resultado é um descendente que herda blocos estruturais de ambos os pais, mantendo a propriedade de permutação válida.

A mutação é implementada na forma de troca de posições (*swap mutation*). Em cada indivíduo selecionado para mutação, duas posições distintas do vetor são sorteadas e seus valores são trocados. Esse operador mantém a natureza permutacional da representação e introduz pequenas perturbações na configuração das rainhas, favorecendo a exploração do espaço de busca sem descaracterizar completamente soluções promissoras.

Cada uma das quatro estratégias de controle populacional atua sobre os mesmos operadores descritos acima, de modo que diferenças de desempenho observadas nos experimentos possam ser atribuídas predominantemente à política de substituição e aos mecanismos associados (como idade máxima, renovação parcial da população ou variação do tamanho populacional).

4.3. Execução dos Experimentos

A ferramenta experimental foi configurada para executar automaticamente todas as combinações de parâmetros definidos na Seção 3. Para cada tamanho de tabuleiro considerado e para cada estratégia de controle populacional, são realizadas múltiplas execuções independentes do algoritmo genético, com sementes diferentes para o gerador de números aleatórios. Essa repetição é essencial para mitigar a variabilidade inerente a métodos estocásticos.

Cada execução segue o fluxo descrito a seguir:

1. **Inicialização**: a partir da configuração fornecida, a população inicial é gerada por embaralhamento aleatório de sequências de 0 a $n-1$. As estruturas internas de controle (como contadores de idade dos indivíduos, quando aplicável) são zeradas.
2. **Laço evolutivo**: em cada geração, a função de aptidão é avaliada para todos os indivíduos (ou para aqueles afetados por modificações, dependendo da estratégia). Em seguida, são aplicados seleção, cruzamento e mutação conforme as taxas configuradas, resultando em novos indivíduos candidatos à substituição na população.
3. **Aplicação da estratégia de controle populacional**: de acordo com o esquema em avaliação, a população é atualizada. No caso da população fixa, o tamanho se mantém constante e os novos indivíduos simplesmente substituem os menos aptos. Na abordagem de elitismo, um subconjunto dos melhores indivíduos é preservado. No controle por idade, a idade de cada indivíduo é incrementada e aqueles que excedem o limite máximo são removidos, permitindo variações controladas no tamanho populacional.
4. **Verificação dos critérios de parada**: após cada geração, verifica-se se algum dos critérios de parada foi satisfeito: (i) solução ótima encontrada; (ii) limite máximo de gerações atingido; ou (iii) limite de tempo excedido. Em qualquer desses casos, a execução é encerrada.
5. **Registro das métricas**: ao final de cada execução, são registrados o tempo total decorrido, o número de gerações percorridas, o critério de parada acionado e um indicador de sucesso, além de informações de configuração (estratégia de controle adotada, tamanho do tabuleiro, semente aleatória, entre outras).

Para facilitar a automação, um script principal recebe como entrada um arquivo de configuração contendo a lista de cenários a serem executados. Cada cenário define parâmetros como tamanho do tabuleiro, estratégia de controle populacional, limites de gerações e de tempo, além do número de repetições independentes. O script percorre essa lista, dispara as execuções correspondentes e grava os resultados em arquivos separados por cenário ou em um único arquivo consolidado, conforme definido na configuração.

4.4. Organização dos Dados e Pré-processamento

Os dados produzidos pela ferramenta experimental são armazenados em arquivos no formato `.csv`, nos quais cada linha corresponde a uma execução do algoritmo genético em um determinado cenário. As colunas incluem, entre outros, os seguintes campos: identificação do cenário, tamanho do tabuleiro, estratégia de controle populacional, semente aleatória, tempo total de execução, número de gerações percorridas, critério de parada acionado e indicador de sucesso.

Antes da etapa de análise estatística, é realizado um pré-processamento desses arquivos com os seguintes objetivos principais:

- **Validação e limpeza**: verificação de registros incompletos ou inconsistentes (por exemplo, execuções interrompidas por falhas externas) e remoção ou correção desses casos, quando necessário;
- **Cálculo de métricas derivadas**: criação de colunas adicionais a partir das métricas brutas, como taxas de sucesso por cenário, tempo médio até a solução, número médio de gerações por execução bem-sucedida e proporção de cada critério de parada acionado;
- **Agrupamento por estratégia e tamanho de problema**: reorganização dos dados em estruturas que facilitem a comparação entre estratégias de controle populacional, mantendo separadas as diferentes dificuldades (valores de n) e consolidando resultados de execuções independentes.

O pré-processamento é implementado em `Python`, utilizando bibliotecas de manipulação de dados para carregar os arquivos `.csv`, aplicar filtros e agregar estatísticas. A partir desse processamento, são gerados conjuntos de dados resumidos que alimentam as tabelas e gráficos discutidos na seção de Resultados e Discussão.

Essa etapa intermediária reduz o acoplamento entre a implementação do algoritmo e a análise dos resultados, permitindo que novos experimentos sejam incorporados ao estudo apenas pela adição de arquivos de saída e pela reexecução dos scripts de processamento, sem alterações no núcleo do algoritmo genético.

5. Resultados e Discussão

Esta seção apresenta e discute os resultados obtidos a partir da execução sistemática dos experimentos descritos na Seção 3 e implementados conforme detalhado na Seção 4. Os dados aqui analisados refletem o comportamento das quatro estratégias de controle populacional avaliadas — **substituição geracional**, **elitismo**, **população fixa simples** e **controle por idade (GAVaPS)** — ao longo de múltiplas execuções independentes do algoritmo genético aplicado ao problema das *n*-rainhas.

A análise foca em cinco aspectos principais:

1. taxa de sucesso em encontrar uma solução válida;
2. número de gerações até a solução;
3. tempo total de execução;
4. distribuição dos critérios de parada (ótimo encontrado, limite de gerações, limite de tempo);
5. impacto do controle por tempo como critério adicional de parada.

Os valores apresentados referem-se às médias, desvios e proporções calculados após o pré-processamento dos arquivos de saída, conforme descrito anteriormente.

5.1. Taxa de Sucesso

A Tabela **??** resume as taxas de sucesso das quatro estratégias para os diferentes tamanhos do tabuleiro considerados. Os resultados mostram que:

- O **elitismo** apresentou, em geral, a maior taxa de sucesso, especialmente para tabuleiros maiores.
- A **substituição geracional** manteve um desempenho estável, porém inferior ao elitismo em cenários mais complexos.
- O esquema de **população fixa simples** teve desempenho equivalente ao geracional em tabuleiros menores, mas reduziu sua taxa de sucesso à medida que n aumentou.
- O **controle por idade (GAVaPS)** demonstrou desempenho competitivo, superando a população fixa em vários tamanhos de tabuleiro e aproximando-se do elitismo nos cenários intermediários.

Esses resultados reforçam achados clássicos na literatura: estratégias que preservam diversidade populacional tendem a ser mais robustas em problemas combinatórios de difícil convergência.

5.2. Número de Gerações até a Solução

Ao analisar o número médio de gerações necessárias para encontrar uma solução, observam-se padrões complementares:

- O **elitismo**, apesar da alta taxa de sucesso, frequentemente precisou de mais gerações para convergir, caracterizando um processo de refinamento contínuo.
- A **substituição geracional** convergiu mais rapidamente em tabuleiros pequenos, mas perdeu eficiência à medida que a complexidade aumentou.
- A **população fixa simples** apresentou alta variação, sugerindo maior sensibilidade a escolhas aleatórias durante a evolução.
- O **GAVaPS** destacou-se por manter um bom equilíbrio entre profundidade de busca e velocidade de convergência, especialmente quando o limite de tempo passou a ser o critério de parada mais frequente.

Essas observações sugerem que o controle por idade ajusta dinamicamente a diversidade da população, evitando convergência prematura sem exigir a preservação rígida de elite.

5.3. Tempo Total de Execução

A inclusão de um **critério de parada baseado em tempo** revelou diferenças significativas entre as estratégias:

- O **controle por idade** foi o menos afetado pelo limite de tempo, indicando boa progressão mesmo em populações mais heterogêneas.
- O **elitismo**, embora eficiente em termos de taxa de sucesso, teve execuções mais longas devido à necessidade de avaliar repetidamente indivíduos elite.
- A **substituição geracional** foi a mais estável, com tempos relativamente previsíveis.
- A **população fixa simples** apresentou encerramentos prematuros com maior frequência, especialmente em instâncias grandes.

Esses resultados mostram que o controle por tempo não apenas evita execuções excessivamente longas, mas também funciona como indicador indireto da "velocidade evolutiva" de cada estratégia.

5.4. Critérios de Parada

As análises mostraram três comportamentos distintos:

1. Em tabuleiros pequenos, a maior parte das execuções parou por **solução encontrada**, independentemente da estratégia.
2. Em tabuleiros intermediários, o elitismo manteve taxas altas de sucesso, enquanto os demais passaram a atingir mais frequentemente o limite de gerações.
3. Em tabuleiros grandes, o **limite de tempo** tornou-se dominante para todas as estratégias, mas o GAVaPS manteve a menor proporção de encerramentos por timeout.

Isso reforça a utilidade do controle por tempo como complemento ao controle por gerações em experimentos de grande porte.

5.5. Desempenho Comparativo Geral

Ao consolidar as métricas, observam-se três conclusões recorrentes:

- O **elitismo** continua sendo o método mais consistente em termos de taxa de sucesso, ao custo de maior tempo de execução.
- A **substituição geracional** e a **população fixa simples** são mais rápidas, porém menos robustas.
- O **controle por idade**, inspirado na abordagem GAVaPS, oferece o melhor compromisso entre diversidade, velocidade e capacidade de adaptação ao limite temporal.

Como resultado, embora nenhuma estratégia seja dominadora em todos os cenários, o controle por idade sobressai como alternativa promissora em problemas combinatórios intensivos, especialmente quando há interesse em limitar o tempo computacional.

5.6. Considerações Sobre o Controle de Tempo

A adição do tempo como critério de parada alterou o equilíbrio entre as estratégias. Em particular:

- Sistemas com **população dinâmica** (como o controle por idade) respondem melhor ao limite temporal, pois a renovação constante evita estagnação.
- Estratégias que preservam explicitamente indivíduos muito aptos sofrem com ciclos mais longos de avaliação.
- O critério de tempo melhora a comparabilidade entre cenários, reduzindo assimetrias na duração dos experimentos.

Essa integração aumenta a utilidade prática dos AGs, aproximando o estudo de condições reais, onde recursos computacionais são limitados.

5.7. Síntese dos Resultados

Com base nas análises, os principais achados são:

- O controle por idade demonstrou excelente equilíbrio entre desempenho e custo computacional.

- O elitismo apresentou as maiores taxas de convergência, porém com maior tempo médio por execução.
- A combinação de limite de gerações e limite de tempo se mostrou fundamental para garantir comparabilidade e previsibilidade.
- Os resultados confirmam que políticas de substituição têm impacto decisivo na eficiência evolutiva de AGs.

Esses aspectos fundamentam as conclusões apresentadas na próxima seção e sustentam o argumento de que diferentes esquemas de controle populacional podem ser mais adequados dependendo das restrições do problema e do ambiente computacional disponível.

6. Conclusão e Trabalhos Futuros

6.1. Conclusões Gerais

Este trabalho investigou o impacto de diferentes estratégias de controle populacional em algoritmos genéticos aplicados ao problema das n-rainhas, incorporando explicitamente um critério de parada baseado em tempo para complementar o limite tradicional de gerações. A partir da implementação modular construída para este estudo, foi possível conduzir experimentos reprodutíveis, automatizados e comparáveis entre si, permitindo analisar o comportamento evolutivo das quatro abordagens avaliadas: substituição geracional, elitismo, população fixa simples e controle por idade (inspirado no GAVaPS).

Os resultados demonstraram que a escolha do mecanismo de controle populacional exerce influência direta tanto na taxa de sucesso quanto na eficiência computacional do algoritmo genético. O elitismo obteve o melhor desempenho médio em termos de soluções encontradas, enquanto o controle por idade destacou-se como a estratégia mais equilibrada, combinando bom nível de diversidade populacional, velocidade de convergência e baixa suscetibilidade ao limite temporal. Em cenários de maior complexidade, esse equilíbrio se tornou especialmente evidente, reforçando a importância de estratégias que permitam renovação contínua da população.

A inclusão do critério de parada por tempo mostrou-se uma decisão essencial para experimentos de maior escala, prevenindo execuções excessivamente longas e revelando diferenças estruturais entre as estratégias. Essa abordagem aproxima o algoritmo genético de situações práticas, em que recursos computacionais são limitados e previsibilidade de execução é desejável.

De modo geral, os experimentos validaram que políticas de substituição não são apenas detalhes operacionais, mas componentes fundamentais da dinâmica evolutiva. A análise conduzida fornece evidências de que o controle por idade, aliado a critérios de parada híbridos, constitui uma alternativa competitiva e flexível para problemas combinatórios intensivos.

6.2. Limitações do Estudo

Apesar dos resultados expressivos, algumas limitações devem ser consideradas:

- **Escopo restrito do problema**: os experimentos foram realizados exclusivamente sobre o problema das n-rainhas, um benchmark importante, porém limitado. Outras classes de problemas podem apresentar comportamentos evolutivos diferentes.

- **Variação controlada de parâmetros**: apenas alguns parâmetros fundamentais (como taxas de mutação, tamanhos populacionais e limites de tempo/gerações) foram explorados. Parâmetros adicionais, como pressão seletiva ou intensidade do elitismo, podem afetar significativamente os resultados.
- **Ausência de aceleração por hardware**: todas as execuções foram realizadas em ambiente controlado e sem paralelização massiva. Estratégias que manipulam populações maiores podem se beneficiar de paralelização, o que não foi considerado.
- **Avaliações estatísticas limitadas**: embora médias e proporções tenham sido analisadas, uma investigação estatística mais aprofundada (intervalos de confiança, testes de hipótese ou análise de variância) não foi conduzida de forma completa.

Essas limitações não comprometem os resultados apresentados, mas indicam caminhos claros para aprofundar a pesquisa.

6.3. Trabalhos Futuros

Com base nas conclusões e limitações identificadas, propõem-se diversas direções para continuidade deste estudo:

- **Aplicar as estratégias a outros problemas**: avaliar as mesmas políticas de substituição em problemas como Caixeiro Viajante, Sudoku, alocação de horários (timetabling) ou otimização contínua.
- **Explorar variantes e ajustes do GAVaPS**: incorporar modelos mais completos da literatura de Michalewicz, como mecanismos avançados de ajuste de tamanho populacional, limites não lineares de idade e pressões adaptativas.
- **Estender a análise estatística**: incluir métodos de inferência que permitam avaliar a significância das diferenças observadas entre as estratégias.
- **Acelerar a implementação**: integrar o algoritmo a bibliotecas de paralelização (como multiprocessing, Numba ou CUDA), permitindo populações maiores e experimentos mais extensos.
- **Investigar estratégias híbridas**: combinar controle por idade com elitismo ou mecanismos adaptativos de mutação para maximizar simultaneamente diversidade e velocidade de convergência.
- **Criar um painel visual interativo**: desenvolver uma ferramenta gráfica para visualizar a evolução das populações, estatísticas de aptidão e comportamento das estratégias em tempo real.

Acredita-se que tais extensões podem enriquecer significativamente o entendimento sobre dinâmicas evolutivas em algoritmos genéticos e ampliar a aplicabilidade prática das estratégias analisadas.“‘

Referências

Ali, H. et al. (2023). Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem. In *Proceedings of ICIT*.

Bell, J. and Stevens, B. (2009). A survey of known results and research areas for n-queens. *Discrete Mathematics*, 309(1):1–31.

de Lacerda, E. G. M. and de Leon Ferreira de Carvalho, A. C. P. (1999). Introdução aos algoritmos genéticos. In *Sistemas inteligentes: aplicações a recursos hídricos e sistemas ambientais*. Ed. Universidade / UFRGS / ABRH, Porto Alegre.

Eiben, A. E. and Smith, J. E. (2015). *Introduction to Evolutionary Computing*. Natural Computing Series. Springer, Berlin Heidelberg, 2 edition.

Farhan, A. S., Tareq, W. Z., and Awad, F. H. (2015). Solving n queen problem using genetic algorithm. *International Journal of Computer Applications*, 122(12).

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Python Software Foundation (2025a). *The Python Language Reference, version 3.x*. Python Software Foundation. Acesso em: 28 jun. 2025.

Python Software Foundation (2025b). *Python Module Reference: random — generate pseudo-random numbers*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Python Software Foundation (2025c). *Python Module Reference: time — Time access and conversions*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Richter, H. and Berlezi, R. (2012). Algoritmo genético aplicado ao problema das n rainhas. *Anais da UNIJUÍ*. Relatório técnico disponível em proceedings.sbmac.

Sarkar, U. and Nag, S. (2017). An adaptive genetic algorithm for solving n-queens problem. *arXiv preprint*.